

TP 9 — Statistiques d'un texte

Semaine 9 ■ Fichiers dans NSI/TP09/ ■ 2 h

Objectifs

- Construire et transformer des listes (boucles et compréhensions)
- Parcourir des chaînes pour en extraire des informations
- Combiner listes et chaînes dans un petit analyseur de texte

Outil du jour : `texte.split()` découpe une chaîne en **liste de mots** : `"le chat dort".split()`
→ `['le', 'chat', 'dort']`.

Partie A — Gammes sur les listes — `listes.py`

(35 min)

Écrire et tester (boucles interdites → compréhension quand c'est indiqué) :

1. `maximum(t)` — le plus grand élément (boucle, sans `max`).
2. `amplitude(t)` — écart entre max et min (réutiliser vos fonctions).
3. `doubles(t)` — nouvelle liste des éléments doublés (**compréhension**).
4. `positifs(t)` — les éléments positifs de `t` (**compréhension avec filtre**).
5. `au-dessus(notes, seuil)` — combien de notes dépassent le seuil ?
6. ★ `normalise(t)` — liste où chaque élément est ramené entre 0 et 1 : $(x - \min) / (\max - \min)$.

Partie B — Gammes sur les chaînes — `chaines.py`

(35 min)

1. `compte(mot, lettre)` — occurrences de la lettre dans le mot.
2. `censure(mot)` — remplace chaque voyelle par "*" (accumulation dans une nouvelle chaîne).
3. `initiales(phrase)` — initiales("Ada Lovelace") → "A.L." (utiliser `.split()`).
4. `mot_le_plus_long(phrase)` — le plus long mot de la phrase.
5. ★ `est_anagramme(m1, m2)` — même nombre de chaque lettre ? (idée simple : comparer `[compte(m1, c) for c in "abc...z"]` pour les deux mots).

Partie C — L'analyseur de texte — `analyse.py`

(40 min)

Copier une vingtaine de lignes d'un texte libre (Wikisource...) dans une variable `texte = """...`. Afficher un petit rapport :

```
=== ANALYSE ===
Caractères : 1042      Mots : 187
Mot le plus long : anticonstitutionnellement
Longueur moyenne des mots : 4.6
Fréquence des voyelles : e: 71  a: 40  i: 33 ...
```

Contraintes : tout passe par **vos fonctions** des parties A et B (les réutiliser, pas les recopier) ; le rapport utilise des f-strings.

★ Bonus : la lettre la plus fréquente du texte (hors espaces) — comparer avec la théorie du TP 6 (le « e » français).

Partie D — Défi ★★

(10 min)

`codes_croissants(mot)` — `True` si les codes Unicode des caractères du mot sont strictement croissants (ex. `"abcx"`). Une seule boucle, comparer `mot[i]` et `mot[i + 1]` — attention à la borne du `range` !